
Analisi dei Requisiti e Architettura di Sistema

Il progetto richiede lo sviluppo di un'applicazione distribuita per un gioco di Trivia Quiz. Il sistema è stato implementato seguendo il modello Client-Server su architettura di rete TCP/IP.

L'implementazione scelta è stata quella di un server concorrente multithread

- **Motivazione:** Questa scelta permette di gestire ogni client tramite un thread dedicato, garantendo che le operazioni bloccanti di I/O (come l'attesa della risposta a una domanda) non interferiscano con gli altri giocatori. Rispetto a un server iterativo, il multithreading offre una gestione parallela delle sessioni di gioco.
- **Analisi Critica:** Il pregio principale risiede nella semplicità del modello di programmazione e nella condivisione dello spazio di indirizzamento, che facilita la gestione della classifica globale. Di contro, l'uso di thread introduce un overhead di memoria per ogni stack allocato; tuttavia, nel contesto di un Trivia Quiz, tale costo è trascurabile rispetto ai vantaggi in termini di latenza.

Il client agisce puramente come interfaccia di input/output: trasmette le scelte dell'utente e visualizza i messaggi ricevuti. Questa scelta centralizza il controllo dei dati (punteggi, validazione nickname) sul server, aumentando la sicurezza e l'integrità del gioco.

Protocollo di Comunicazione e Scambio Dati

La comunicazione avviene tramite socket di tipo STREAM (TCP). La scelta del protocollo TCP è fondamentale per garantire che le domande e, soprattutto, le risposte arrivino integre e nell'ordine corretto, evitando i problemi di perdita di pacchetti tipici di UDP.

Per rispettare i requisiti di sistema e gestire stringhe di lunghezza variabile, è stato implementato un protocollo di messaggistica esplicito:

Prima di ogni invio, il mittente trasmette un intero a 32 bit che rappresenta la lunghezza in byte del messaggio. Viene utilizzata la funzione `htonl()` per convertire il dato nel *network byte order*, garantendo l'interoperabilità tra architetture diverse. Successivamente, viene inviato il corpo del messaggio (stringa).

Il ricevente legge i primi 4 byte tramite `recv()`, converte la lunghezza con `ntohl()` e procede alla lettura del numero esatto di byte rimanenti.

Questo approccio risolve il problema della frammentazione del flusso TCP, assicurando che il ricevente sappia esattamente dove termina un messaggio e inizi il successivo. Rispetto a un protocollo a lunghezza fissa, questo metodo ottimizza l'uso della banda.

Gestione dei Dati e Sincronizzazione

Il server gestisce i partecipanti tramite una lista concatenata dinamica di strutture `dati_giocatori`. Ogni nodo memorizza il nickname, i punteggi per i due temi e lo stato di completamento dei quiz.

A differenza di una pre-caricamento totale in memoria, il server accede ai file `domande.txt` e `risposte.txt` in fase di esecuzione. Questo approccio riduce l'impronta di memoria iniziale del server, sebbene richieda operazioni di I/O su disco durante il gioco.

Poiché la lista dei giocatori e il contatore utenti_connessi sono risorse condivise tra tutti i thread, è stato implementato un meccanismo di sincronizzazione tramite Mutex (`pthread_mutex_t`).

- La procedura di registrazione del nickname avviene in sezione critica: il server scorre la lista per verificare l'univocità del nome prima di inserire il nuovo nodo.
- L'uso del lock garantisce la coerenza dei dati (evitando che due utenti prendano lo stesso nome contemporaneamente), ma è stato limitato alle sole operazioni strettamente necessarie per non penalizzare le prestazioni complessive.

Analisi Critica: Pregi e Difetti

IL server gestisce correttamente le disconnessioni improvvise. Se un client chiude il socket, il thread rileva il valore di ritorno ≤ 0 della `recv`, rimuove l'utente dalla classifica in mutua esclusione, libera la memoria e chiude il descrittore.

Flessibilità della Classifica: L'uso di un array di appoggio per l'ordinamento (Bubble Sort) permette di mostrare i dati in modo decrescente senza dover mantenere la lista concatenata costantemente ordinata, semplificando gli inserimenti.

Difetti e Sviluppi Futuri

Scalabilità dell'I/O: L'apertura dei file per ogni quiz potrebbe diventare un collo di bottiglia in presenza di centinaia di richieste simultanee.

Sicurezza: Il protocollo non prevede crittografia. In un contesto reale, sarebbe opportuno integrare TLS/SSL per proteggere i dati in transito